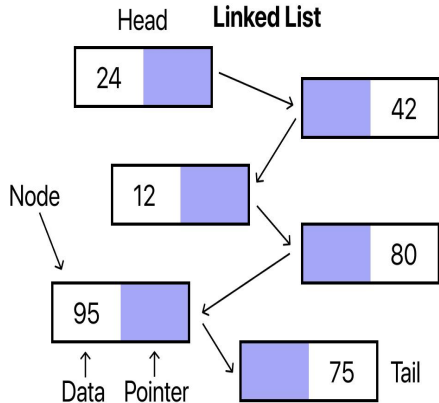
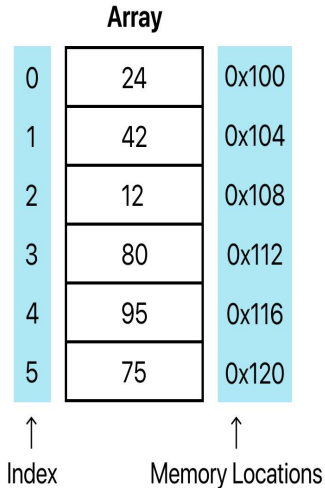


Listas enlazadas sencillas

Linked Lists



Implementación

<https://github.com/leidy-m/estructuras-datos/blob/main/EjemplosClases/ListaEnlazada.py>

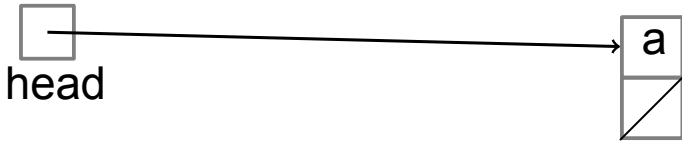
- Nodo, ListaEnlazada

Stack Implementation with Linked List



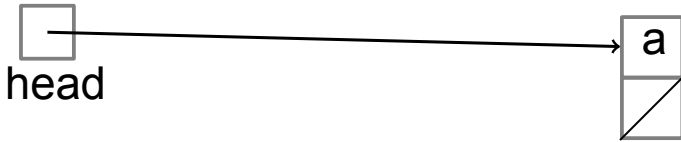
Push (a)

Stack Implementation with Linked List



Push (a)

Stack Implementation with Linked List

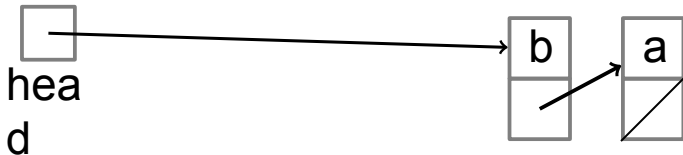


Stack Implementation with Linked List



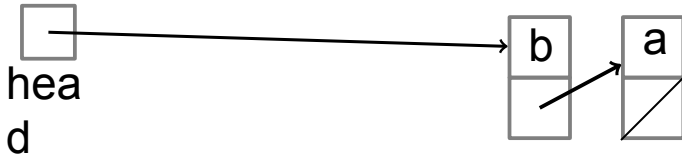
Push (b)

Stack Implementation with Linked List

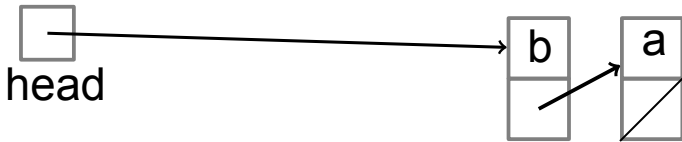


Push (b)

Stack Implementation with Linked List

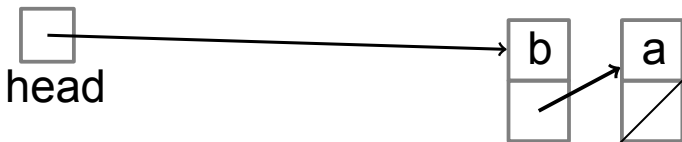


Stack Implementation with Linked List



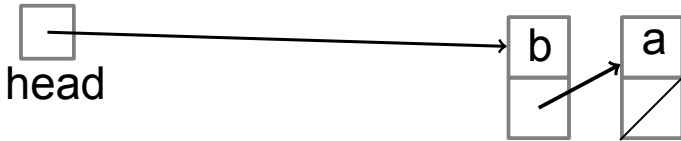
Top ()

Stack Implementation with Linked List

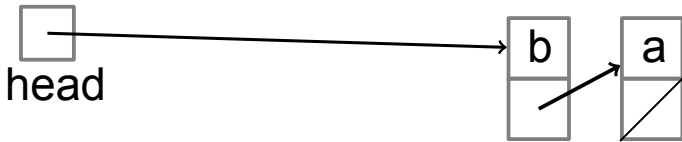


$\text{Top}() \rightarrow b$

Stack Implementation with Linked List

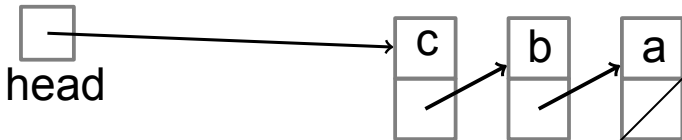


Stack Implementation with Linked List



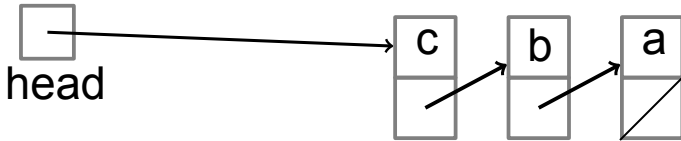
Push (c)

Stack Implementation with Linked List

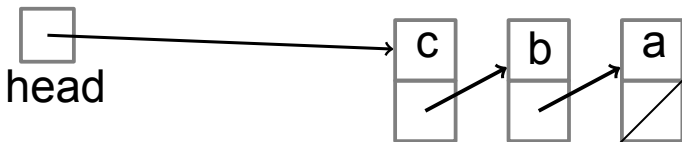


Push(c)

Stack Implementation with Linked List

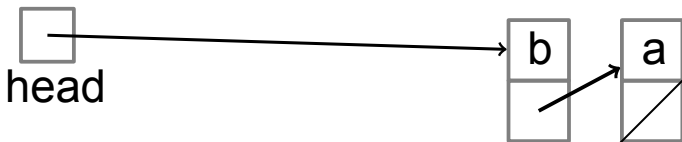


Stack Implementation with Linked List



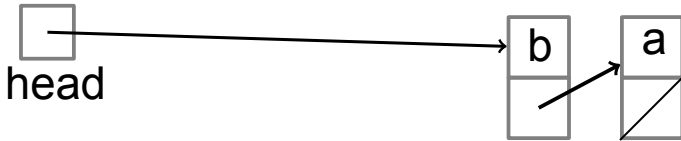
Pop ()

Stack Implementation with Linked List

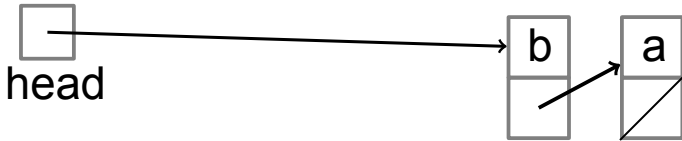


Pop () $\rightarrow$ c

Stack Implementation with Linked List

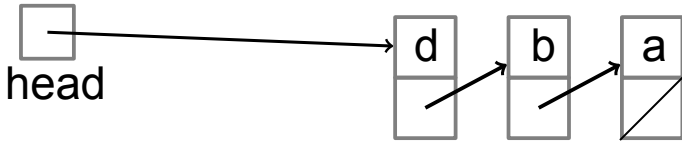


Stack Implementation with Linked List



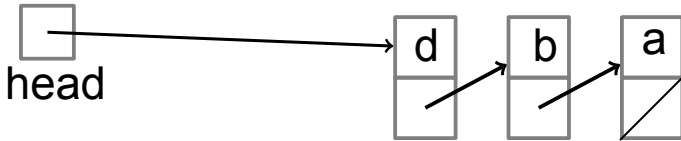
Push (d)

Stack Implementation with Linked List

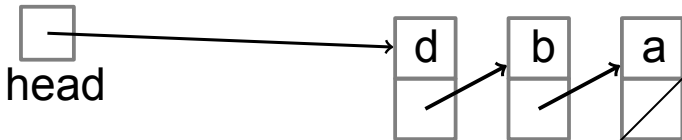


Push (d)

Stack Implementation with Linked List

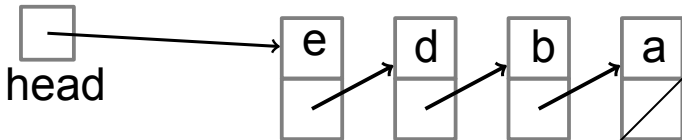


Stack Implementation with Linked List



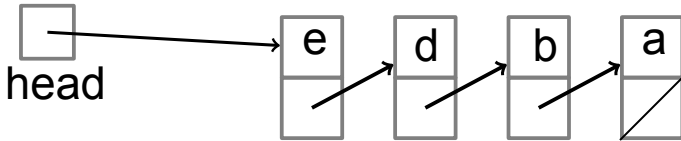
Push (e)

Stack Implementation with Linked List

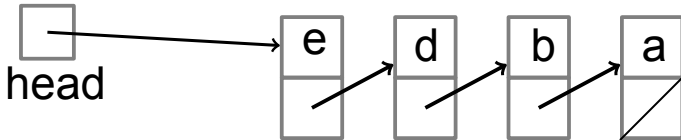


Push (e)

Stack Implementation with Linked List

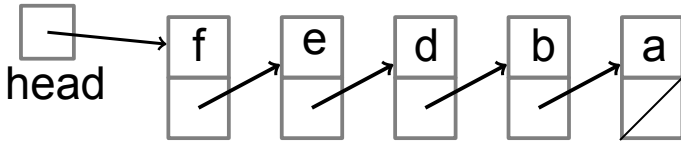


Stack Implementation with Linked List



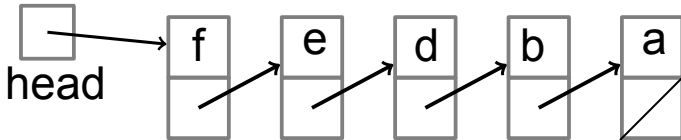
Push (f)

Stack Implementation with Linked List

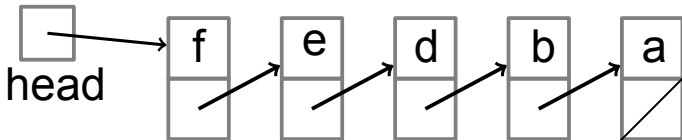


Push(f)

Stack Implementation with Linked List

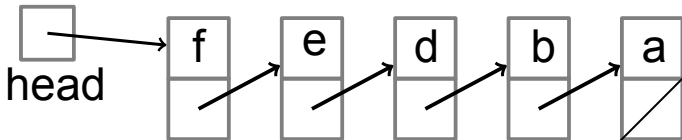


Stack Implementation with Linked List



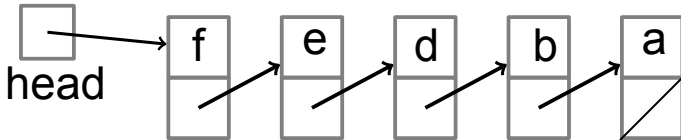
Empty ()

Stack Implementation with Linked List

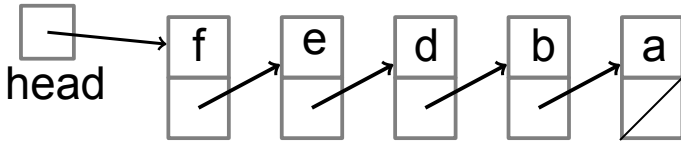


`Empty ()` $\rightarrow$ `False`

Stack Implementation with Linked List

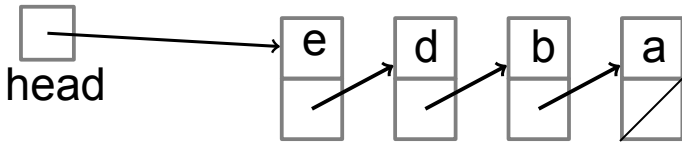


Stack Implementation with Linked List



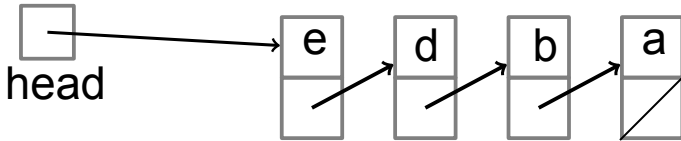
Pop()

Stack Implementation with Linked List

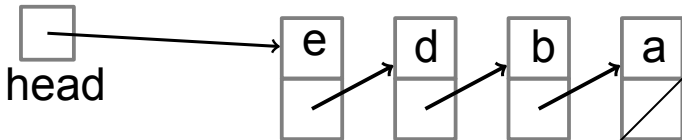


Pop() → f

Stack Implementation with Linked List

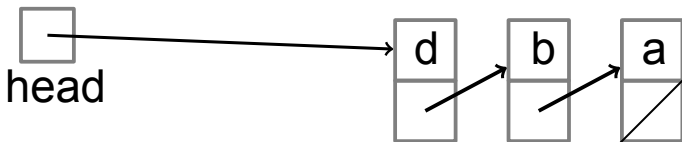


Stack Implementation with Linked List



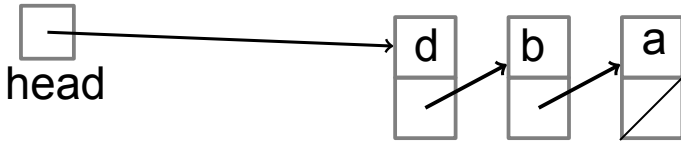
Pop ()

Stack Implementation with Linked List

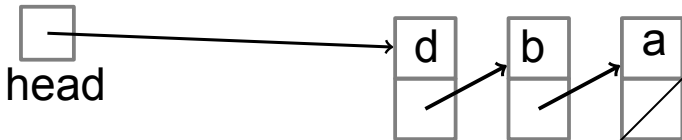


Pop () $\rightarrow$ e

Stack Implementation with Linked List

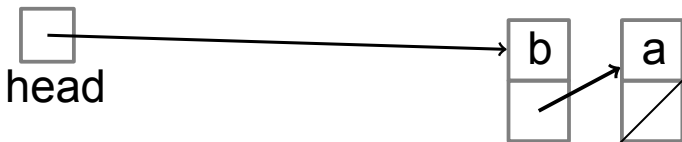


Stack Implementation with Linked List



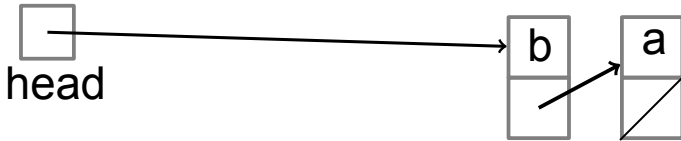
Pop ()

Stack Implementation with Linked List



Pop () $\rightarrow$ d

Stack Implementation with Linked List

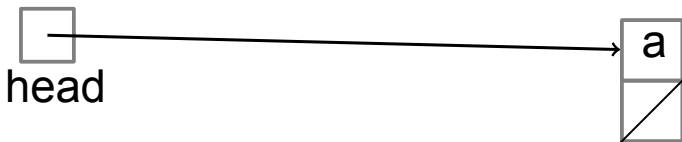


Stack Implementation with Linked List



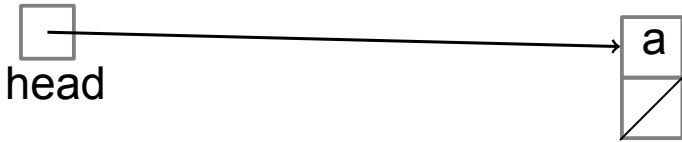
Pop ()

Stack Implementation with Linked List

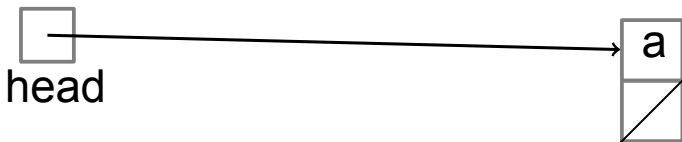


Pop () $\rightarrow$ b

Stack Implementation with Linked List



Stack Implementation with Linked List



Pop ()

Stack Implementation with Linked List



Pop () $\rightarrow$ a

Stack Implementation with Linked List



head

Stack Implementation with Linked List



Empty ()

Stack Implementation with Linked List



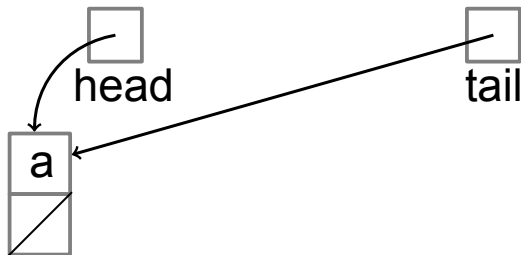
$\text{Empty}() \rightarrow \text{True}$

Summary

- Stacks can be implemented with either an array or a linked list.
- Each stack operation is $O(1)$: Push, Pop, Top, Empty.
- Stacks are occasionally known as LIFO queues.

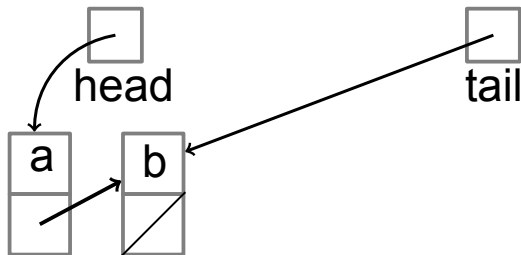
Last In - First Out

Queue Implementation with Linked List



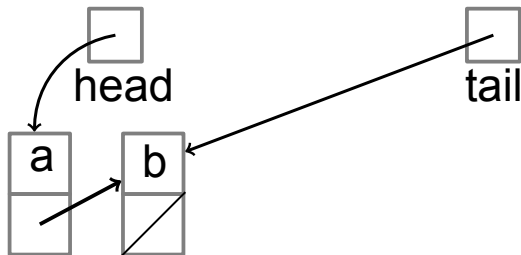
Enqueue (a)

Queue Implementation with Linked List



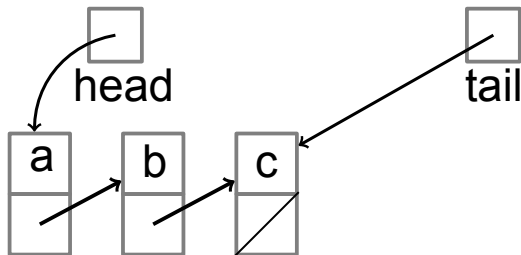
Enqueue (b)

Queue Implementation with Linked List



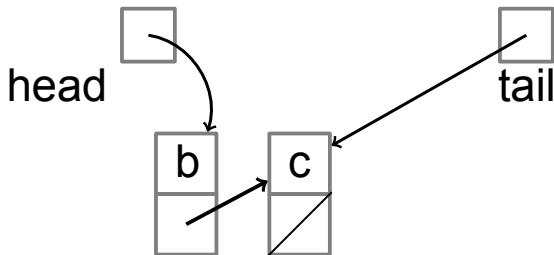
`Empty ()` $\rightarrow$ `False`

Queue Implementation with Linked List



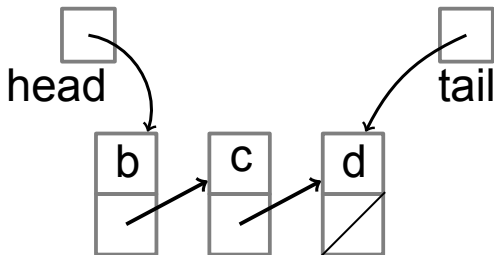
Enqueue (c)

Queue Implementation with Linked List



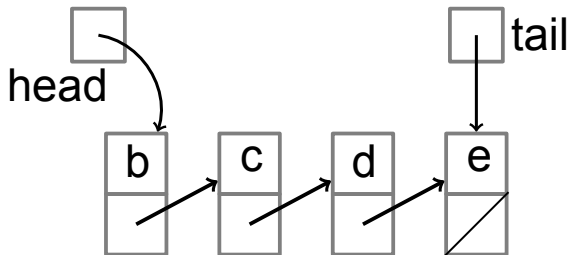
Dequeue () $\rightarrow$ a

Queue Implementation with Linked List



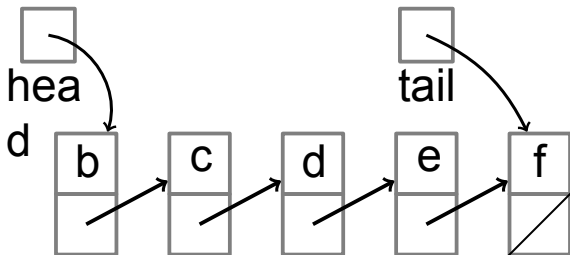
Enqueue (d)

Queue Implementation with Linked List



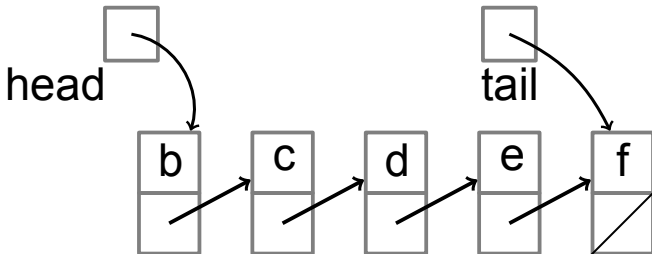
Enqueue(e)

Queue Implementation with Linked List

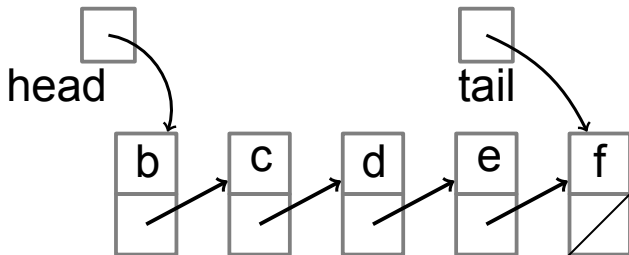


Enqueue (f)

Queue Implementation with Linked List

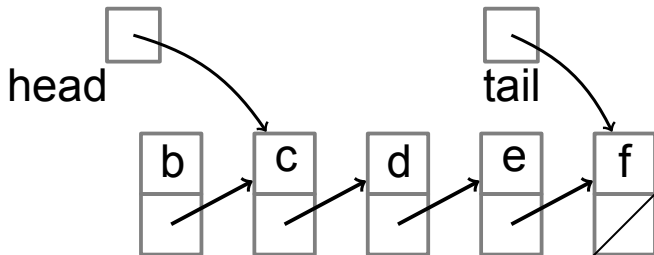


Queue Implementation with Linked List



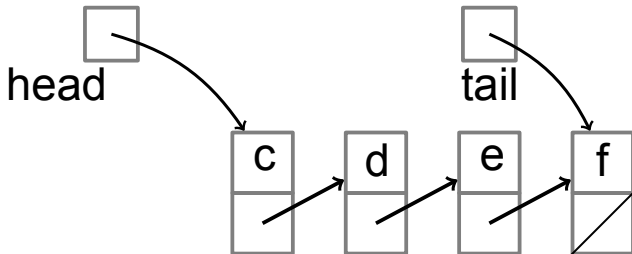
Dequeue()

Queue Implementation with Linked List

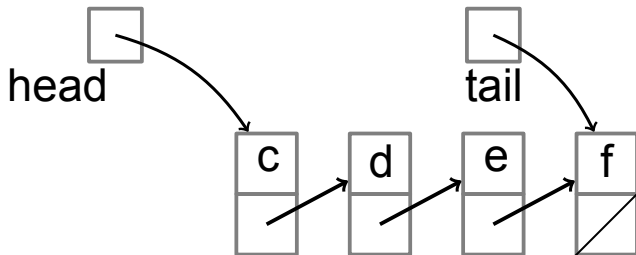


Dequeue() → b

Queue Implementation with Linked List

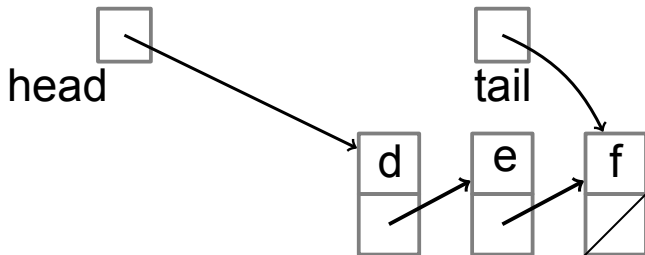


Queue Implementation with Linked List



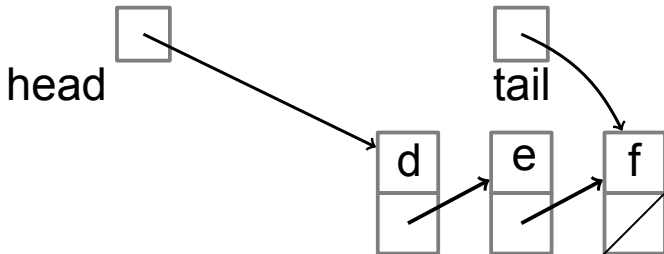
Dequeue ()

Queue Implementation with Linked List

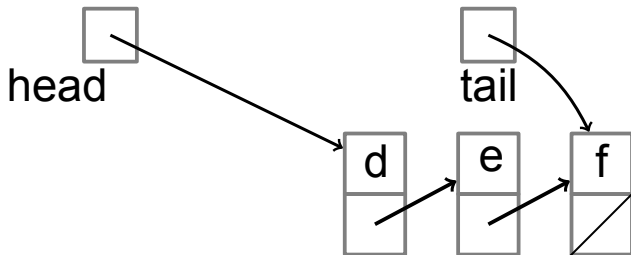


Dequeue () $\rightarrow$ c

Queue Implementation with Linked List

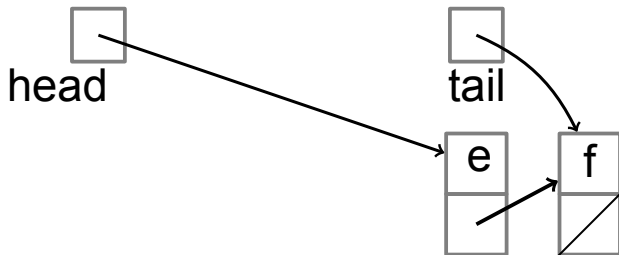


Queue Implementation with Linked List



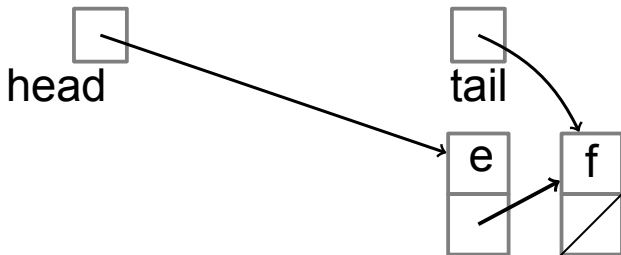
Dequeue ()

Queue Implementation with Linked List

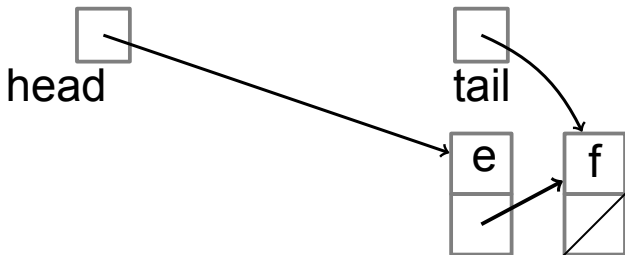


Dequeue () $\rightarrow$ d

Queue Implementation with Linked List

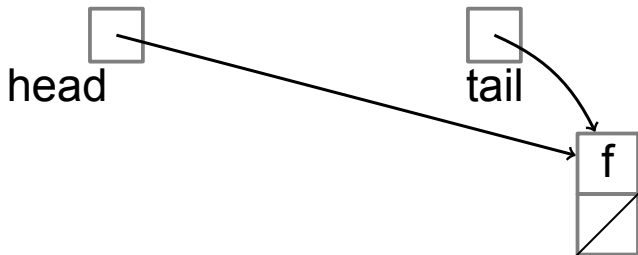


Queue Implementation with Linked List



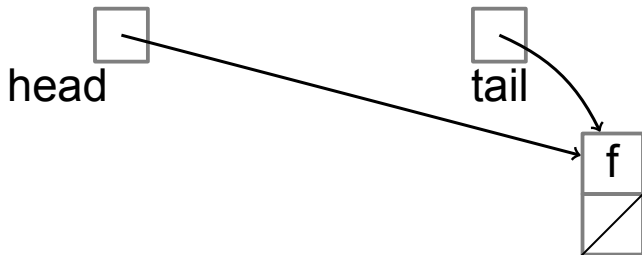
Dequeue ()

Queue Implementation with Linked List

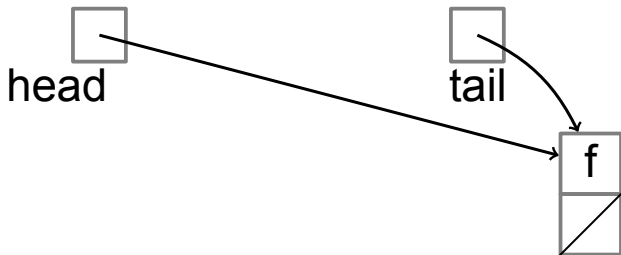


Dequeue () $\rightarrow$ e

Queue Implementation with Linked List



Queue Implementation with Linked List



Dequeue ()

Queue Implementation with Linked List



head



tail

Dequeue () $\rightarrow$ f

Queue Implementation with Linked List



head



tail

Queue Implementation with Linked List



head



tail

Empty ()

Queue Implementation with Linked List



head



tail

`Empty ()` $\rightarrow$ `True`

Queue Implementation with Linked List



head



tail

Summary

- Queues can be implemented with either a linked list (with tail pointer) or an array.
- Each queue operation is $O(1)$: Enqueue, Dequeue, Empty.

Queue Implementation with Linked List

- Enqueue: **use** List.PushBack
- Dequeue: **use** List.TopFront
and List.PopFront
- Empty: **use** List.Empty